39

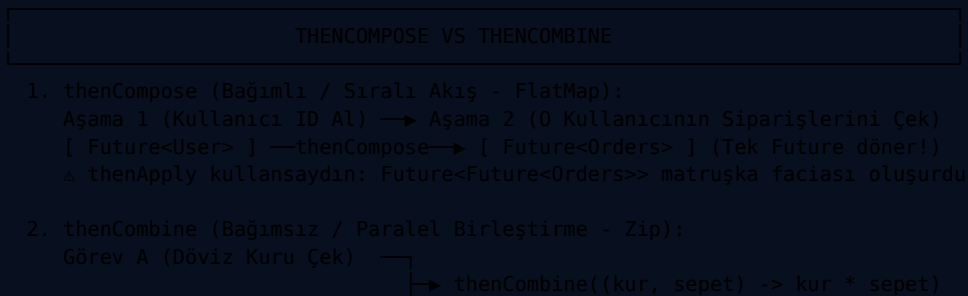
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Bileşim ve Birleştirme: thenCompose (FlatMap) vs thenCombine (Zip) Farkı

Composition & Merging: thenCompose (FlatMap) vs thenCombine (Zip)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

thenCompose: Bir dedektif gibidir. Önce anahtarın nerede olduğunu bulur (1. aşama), sonra o anahtarla kasayı açar (2. aşama). Biri bitmeden diğeri başlayamaz; iki adımı tek bir temiz sonuca bağlar. thenCombine ise düğün pastası hazırlamak gibidir: Biri pastayı fırınlarken diğeri süsleme kremasını çırpar (iki bağımsız iş aynı anda çalışır). İkisi de bitince pastayla kremayı birleştirirler!

🇬🇧 ENGLISH (ELI5)

`thenCompose` is sequential dependency (monadic flatMap): Step 1 retrieves the encryption key; Step 2 uses that key to decrypt the vault payload. Step 2 cannot begin until Step 1 yields its output. `thenCombine` is parallel aggregation (applicative zip): Worker A calculates flight pricing while Worker B fetches hotel availability concurrently. When both arrive, their results are combined into a travel package.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Monadik Yapı ve Tip İmzaları: 1) `thenCompose(Function\<T, CompletionStage\<U>>)`: Fonksiyonel programlamadaki `flatMap` operatörünün birebir karşılığıdır. Eğer `thenApply` kullansaydınız `CompletableFuture\<CompletableFuture\<U>>` şeklinde içiçe geçmiş (nested) bir tür elde ederdiniz. `thenCompose` bu katmanı düzleştirir (flattening). 2) `thenCombine(CompletionStage\<U>, BiFunction\<T, U, V>)`: İki bağımsız `CompletableFuture`'ı paralel olarak koşturur. Her ikisi de başarıyla tamamlandığında `BiFunction`'ı tetikleyerek iki sonucu tek bir `V` değerinde sentezler. Biri çökerse birleşik future da çöker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Monadic Anatomy: 1) `thenCompose`: Represents functional `flatMap`. If you map an asynchronous function `T -> CompletableFuture\<U>` via `thenApply`, you produce a nested `CompletableFuture\<CompletableFuture\<U>>`. `thenCompose` flattens the hierarchy, binding stages sequentially. 2) `thenCombine`: Functional `zipWith`. Executes both stages asynchronously in parallel. Upon dual completion, it supplies both outputs `(T, U)` to a `BiFunction\<T, U, V>`. If either stage aborts exceptionally, the aggregated stage terminates with `ExecutionException`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: thenCompose (Sıralı) ve thenCombine (Paralel) Kullanımı
import java.util.concurrent.CompletableFuture;

public class CompositionAndCombineDemo {
    record User(long id, String creditTier) {}
    record CreditLimit(long userId, double limitUsd) {}
    record Cart(long userId, double totalUsd) {}
    record CheckoutDecision(boolean approved, String reason) {}

    static CompletableFuture<User> fetchUser(long id) {
        return CompletableFuture.supplyAsync(() -> new User(id, "PREMIUM"));
    }

    static CompletableFuture<CreditLimit> fetchLimit(User user) {
        return CompletableFuture.supplyAsync(() ->
            new CreditLimit(user.id(), user.creditTier().equals("PREMIUM") ? 10000.0 : 1000.0)
        );
    }

    static CompletableFuture<Cart> fetchCart(long userId) {
        return CompletableFuture.supplyAsync(() -> new Cart(userId, 3200.0));
    }

    void main() {
        long userId = 101L;

        // 1. thenCompose: User -> CreditLimit (Bağımlı Sıralı Akış)
        CompletableFuture<CreditLimit> limitFuture = fetchUser(userId)
            .thenCompose(CompositionAndCombineDemo::fetchLimit);

        // 2. Sepeti bağımsız paralel çek
        CompletableFuture<Cart> cartFuture = fetchCart(userId);

        // 3. thenCombine: Limit ile Sepeti birleştirip karar ver (Paralel Zip!)
        CompletableFuture<CheckoutDecision> decisionFuture = limitFuture.thenCombine(cartFuture,
            (limit, cart) -> {
                boolean ok = cart.totalUsd() <= limit.limitUsd();
                return new CheckoutDecision(ok, ok ? "Onaylandı" : "Kredi limiti yetersiz!");
            });

        CheckoutDecision decision = decisionFuture.join();
        System.out.println("Ödeme Kararı: " + decision);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenCombine kullanırken iki görevin birbirini beklemediğinden emin olun. Eğer Görev A'nın girdisi Görev B'ye lazımsa thenCombine ÇALIŞMAZ; o zaman mecburen thenCompose kullanmalısınız. Yanlış operatör seçimi deadlock veya derleme hatalarına yol açar.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`thenCombine` mandates that both futures are strictly independent. If stage B requires an argument generated by stage A, using `thenCombine` is invalid; you must use `thenCompose`. Misusing them creates cascading compile-time type mismatches.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenCompose bağımlı asenkron adımları birbirine zincirler (flatMap). thenCombine bağımsız iki asenkron görevi paralel çalıştırıp birleştirir (zip).

🎯 Key Takeaways for the Interview

• `thenCompose` sequences dependent async stages (flatMap). `thenCombine` executes two independent stages in parallel and fuses their outputs (zip).